

Tower of Hanoi using Recursion

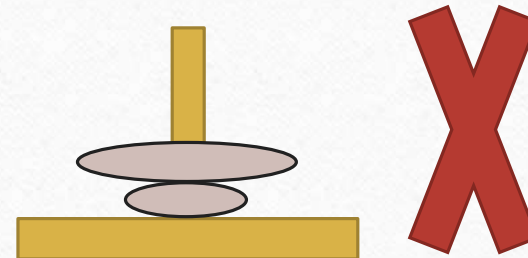
What is Recursion?

- When a function calls itself, it's called Recursion
- Recursion is useful in solving problems which can be broken down into smaller problems of the **same kind**.
- For Example:

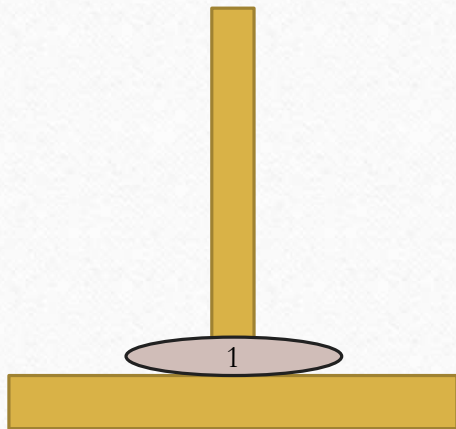
```
1 function factorial(x)
2   if x is 1           // base case
3     return 1
4   return x*factorial(x-1) // break into smaller problem(s)
```

What is Tower of Hanoi problem?

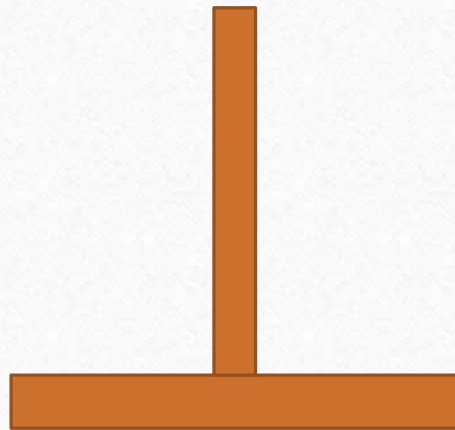
- Tower of Hanoi consists of three pegs or towers with n disks placed one over the other.
- The objective of the puzzle is to move the stack to another peg following these simple rules.
 - Only one disk can be moved at a time.
 - No disk can be placed on top of the smaller disk



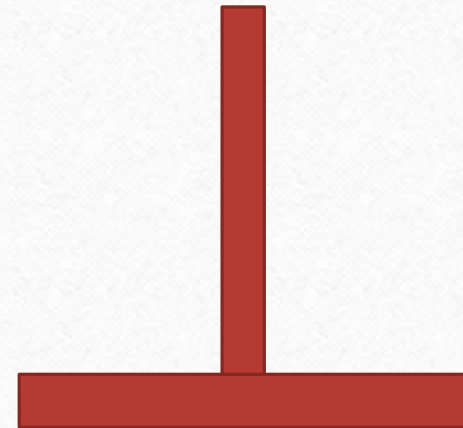
Tower of Hanoi for 1 disk



A
(from)

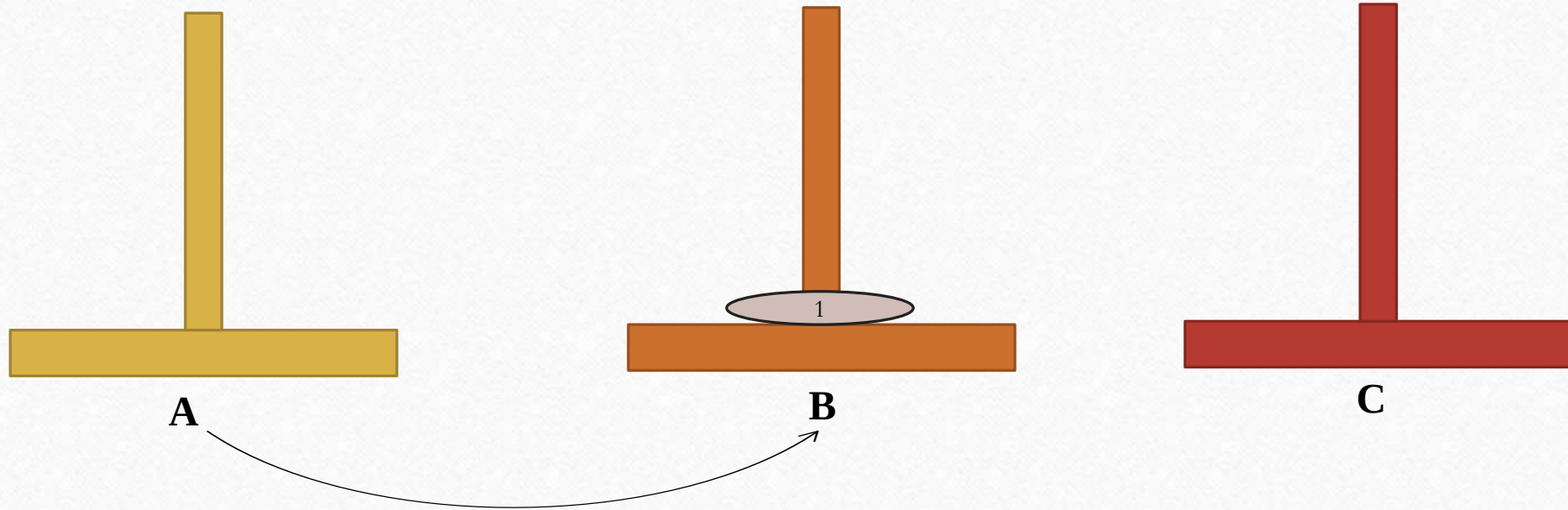


B
(to)

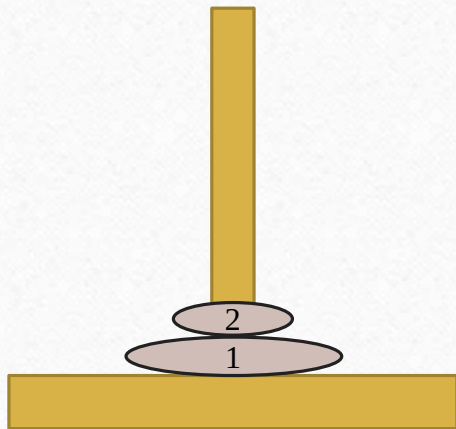


C
(using)

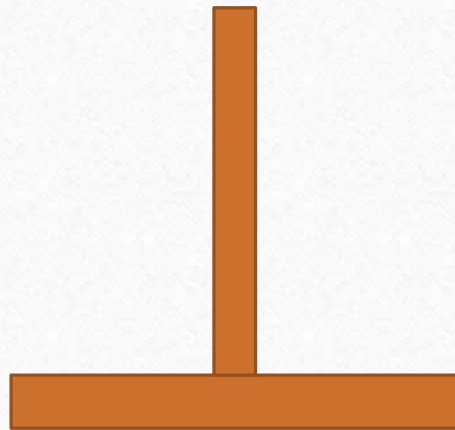
Step 1 Move A to B



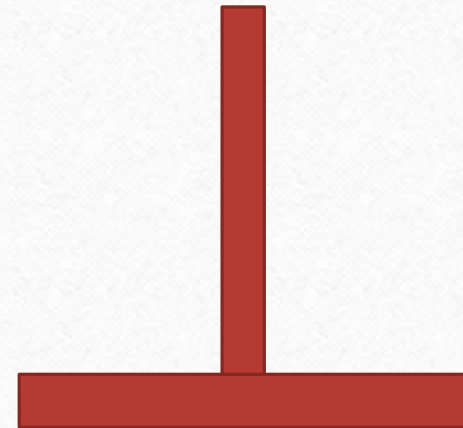
Tower of Hanoi for 2 disks



A
(from)

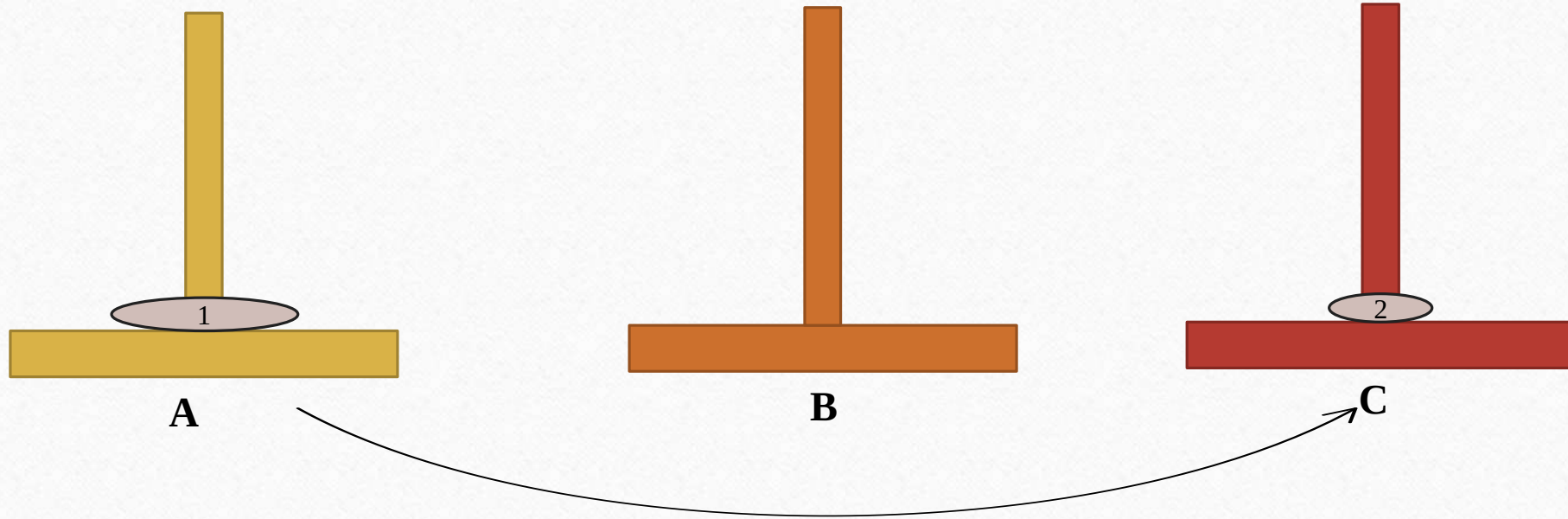


B
(to)

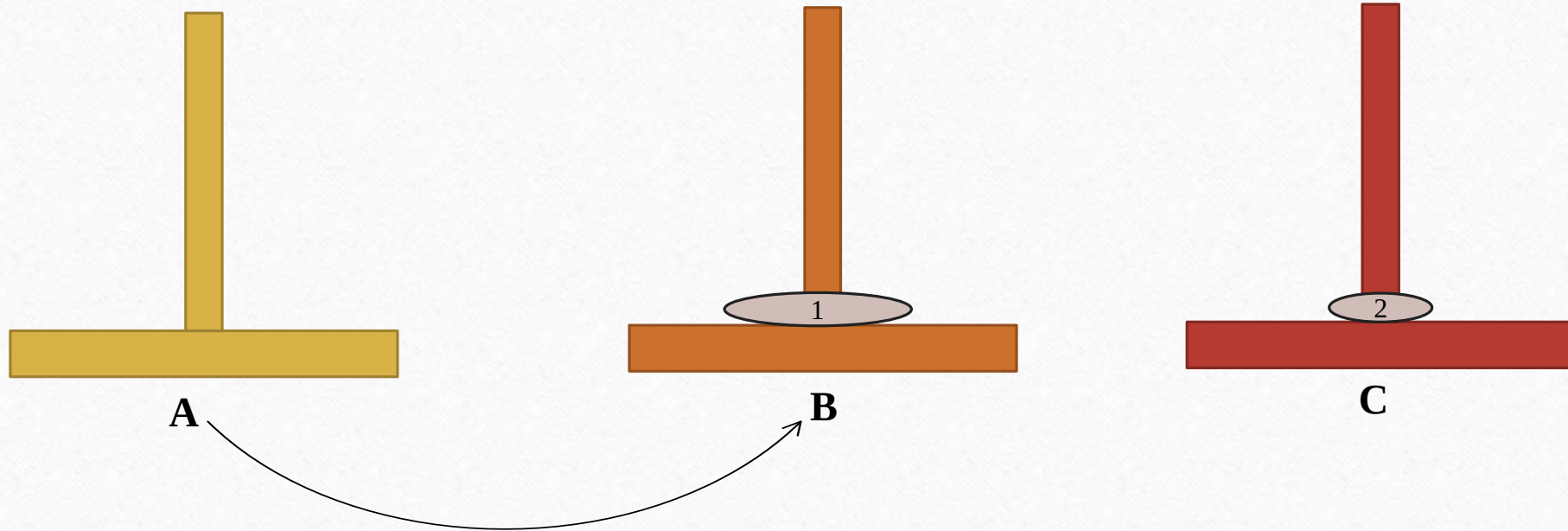


C
(using)

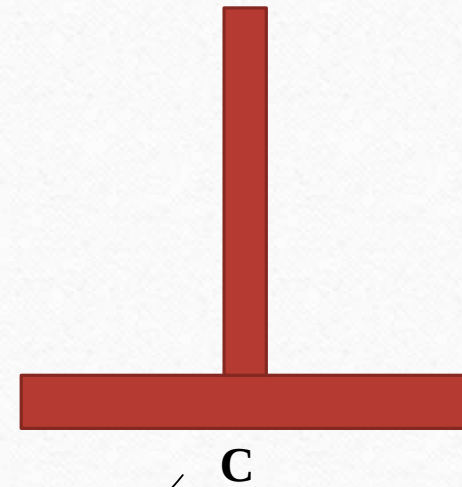
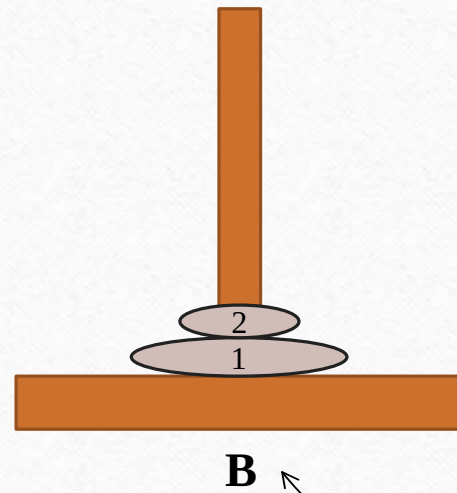
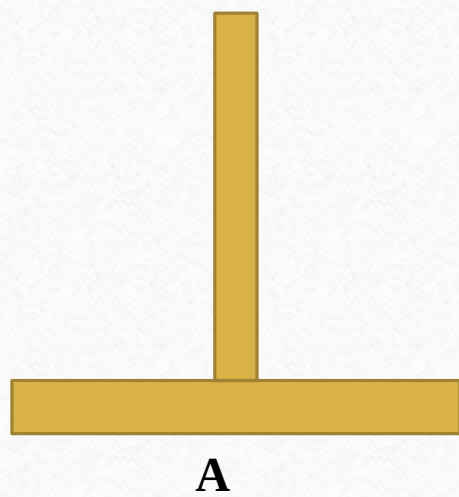
Step 1 Move A to C



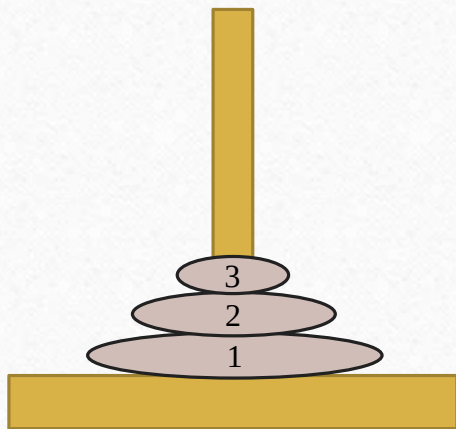
Step 2 Move A to B



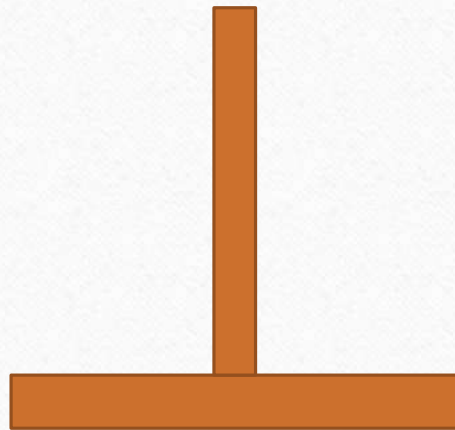
Step 3 Move C to B



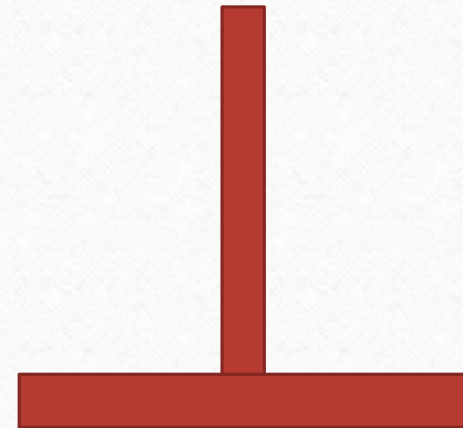
Tower of Hanoi for 3 disks



A
(from)

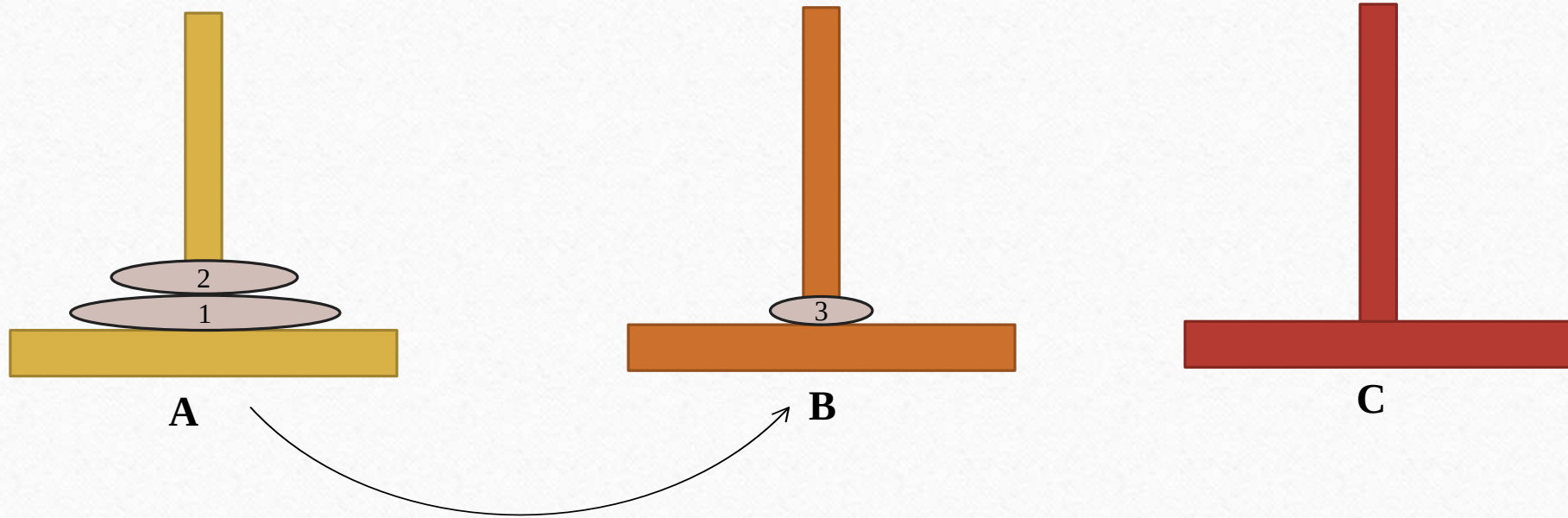


B
(to)

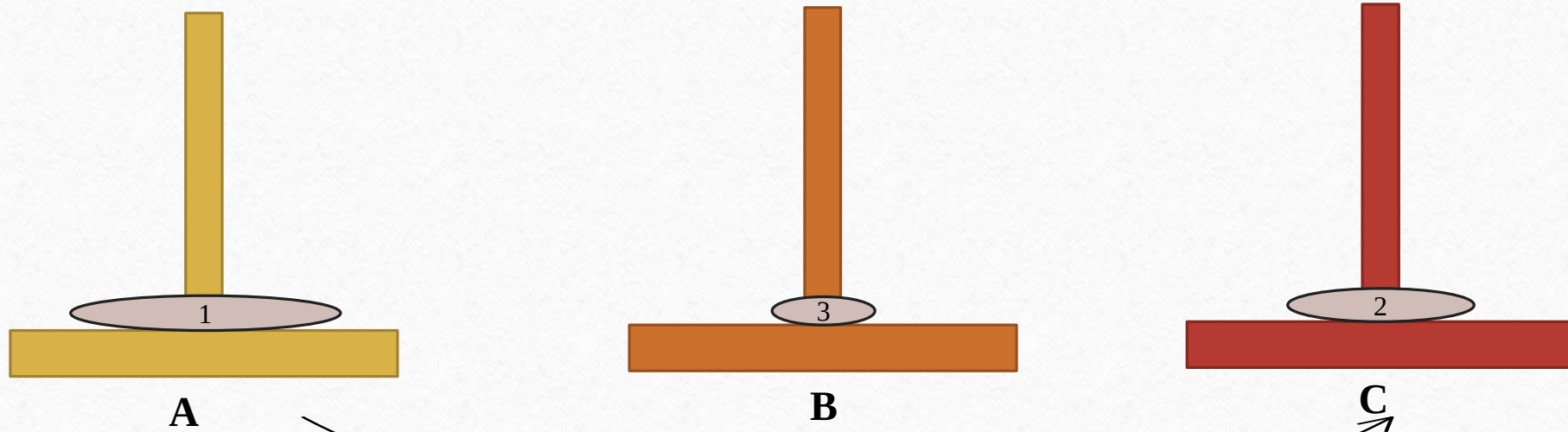


C
(using)

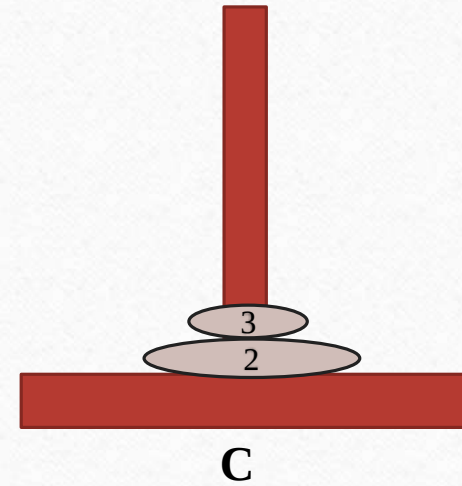
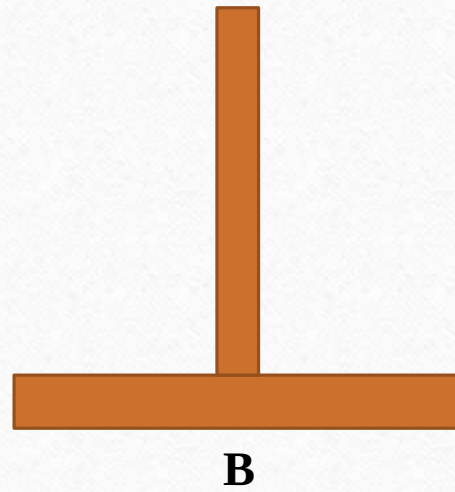
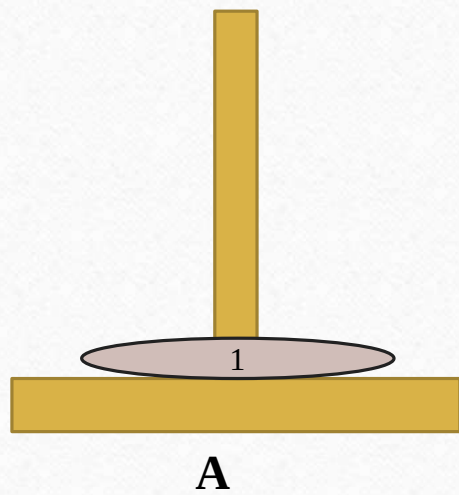
Step 1 Move A to B



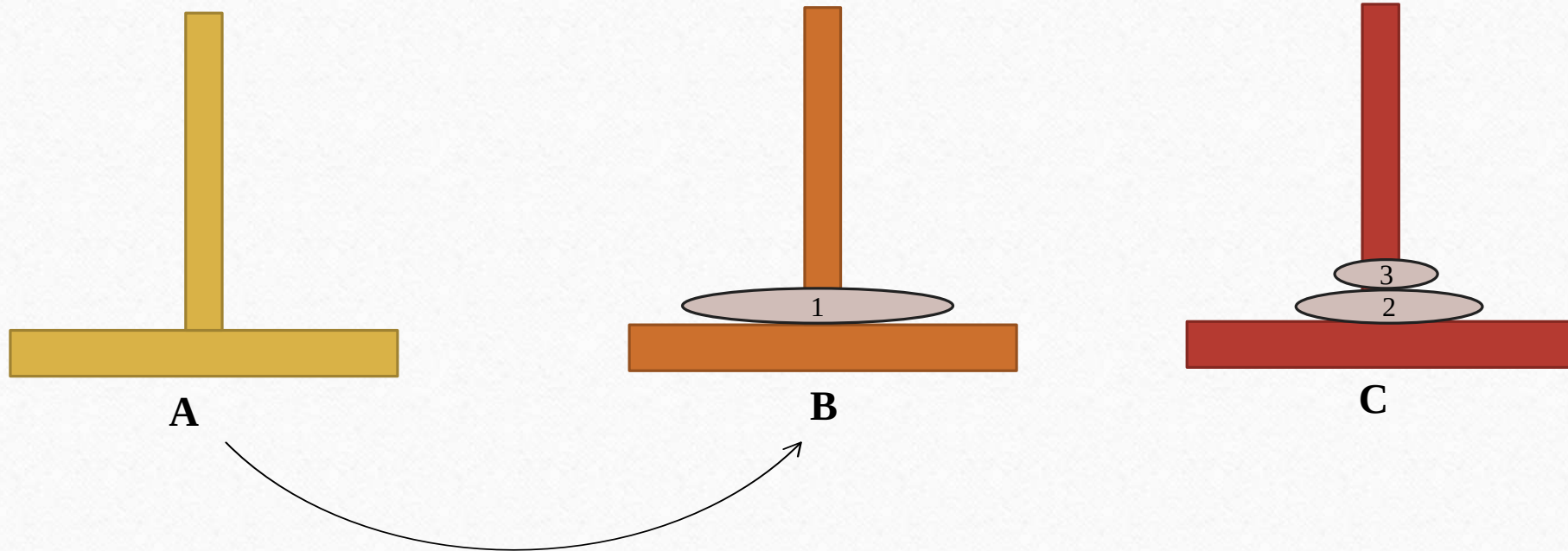
Step 2 Move A to C



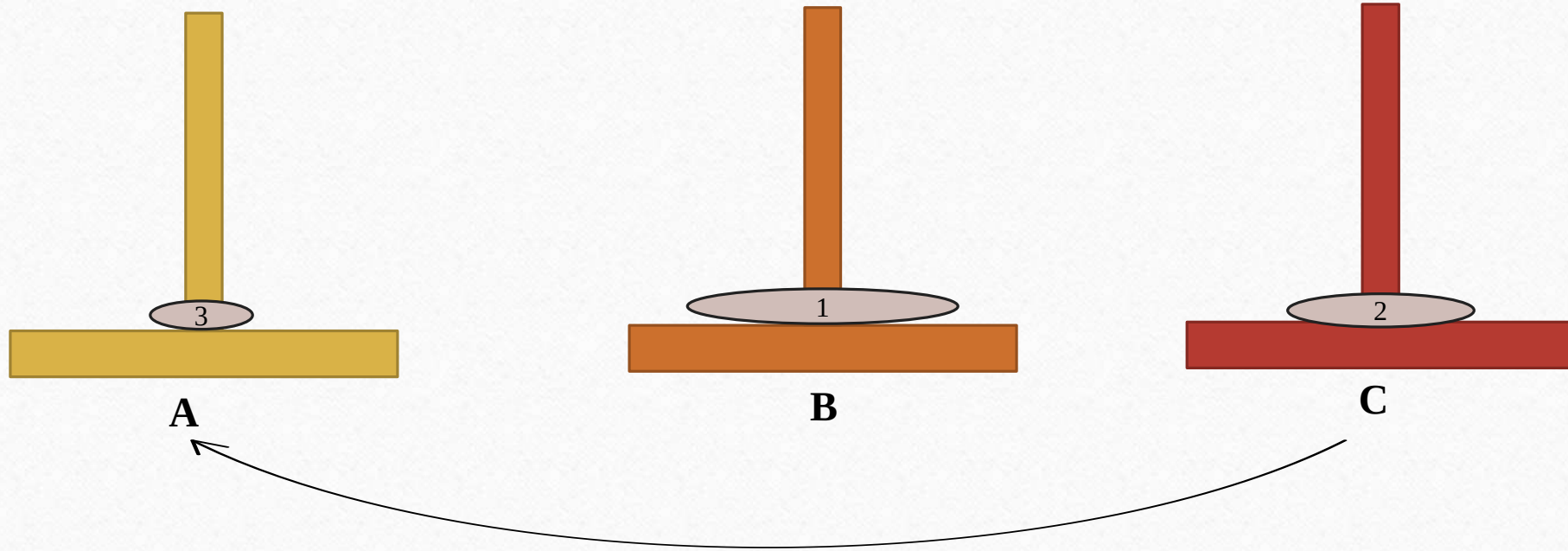
Step 3 Move B to C



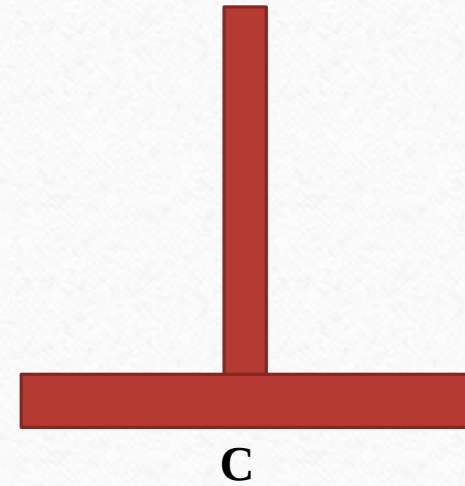
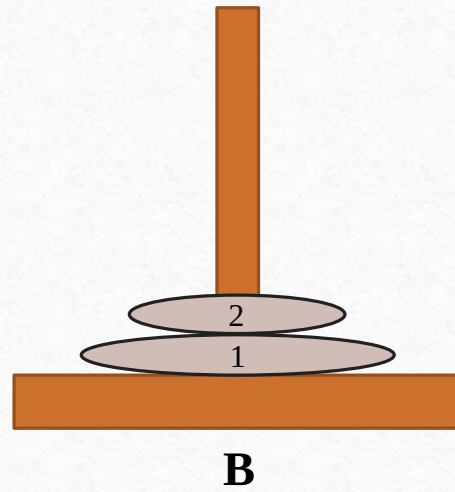
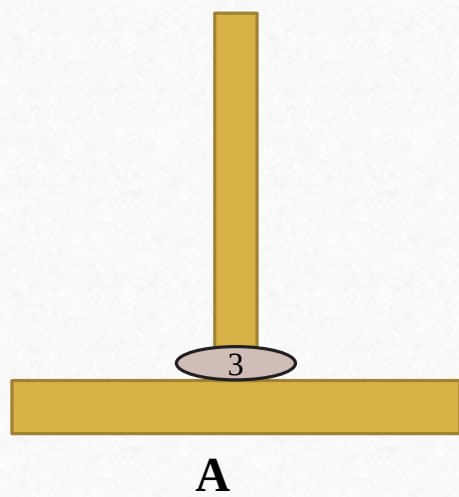
Step 4 Move A to B



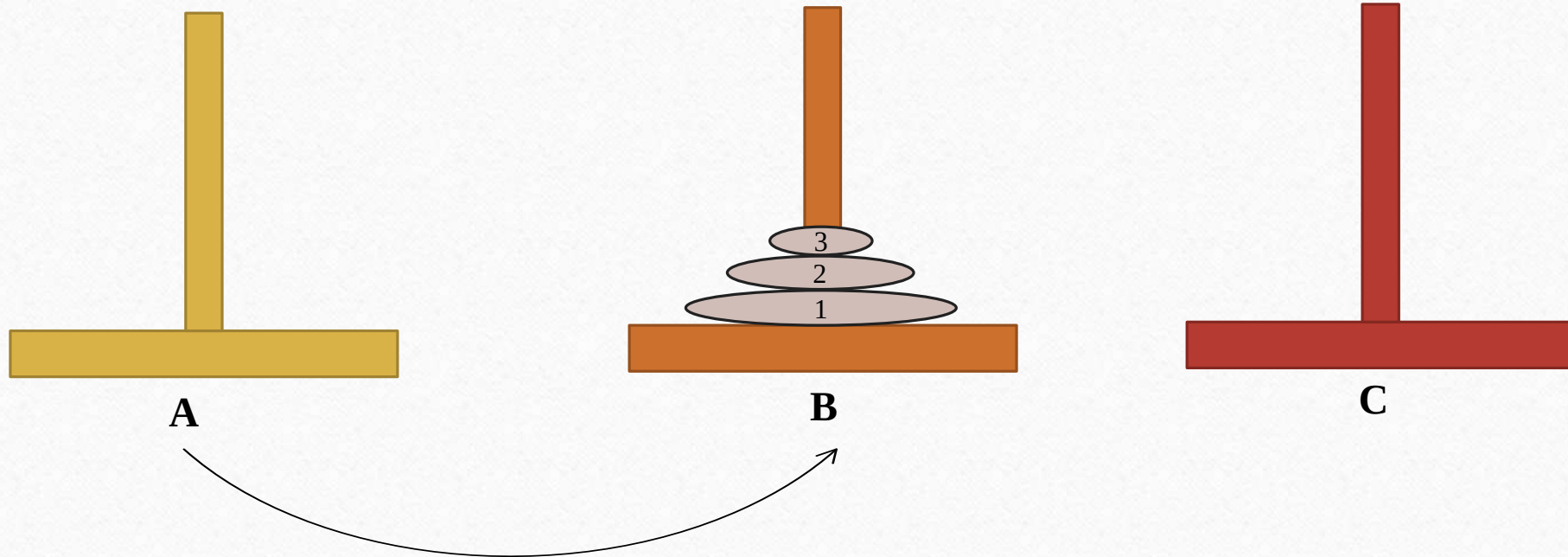
Step 5 Move C to A



Step 6 Move C to B



Step 7 Move A to B



Solution Idea

- (*basis*) One ring can always be moved.
- (*inductive step*) Assume that we can move $n-1$ disks from any given tower to any other tower. Since any of the disks 1 through $n-1$ can be placed on top of disk n , all n rings can be moved as follows:
 1. Move $n-1$ rings from tower A to tower C.
 2. Move ring n from tower A to tower B.
 3. Move $n-1$ rings from tower C to tower B.

Recursive Algorithm

- The procedure TOWER(n, A, B, C) moves n disks from Tower A to Tower B :
procedure TOWER(n, A, B, C)

if ($n == 1$) / base case */*

Move(A, B);

else { / recursion */*

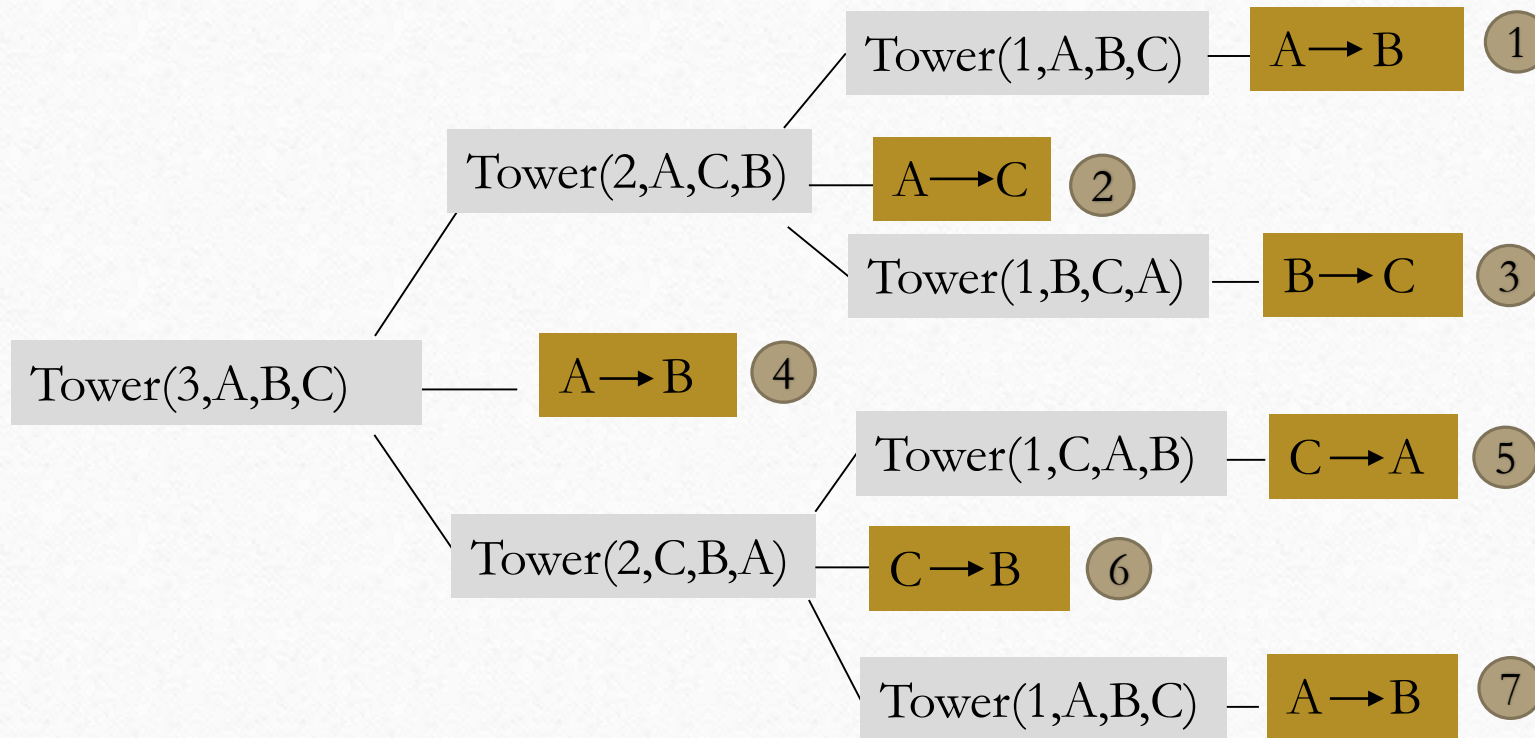
TOWER ($n-1, A, C, B$);

Move(A, B);

TOWER ($n-1, C, B, A$)

}

Recursive Calls for n=3



```
procedure TOWER(n, A,B,C)
  if (n == 1) /* base case */
    Move( A, B );
  else { /* recursion */
    TOWER ( n-1, A, C, B );
    Move( A, B );
    TOWER ( n-1, C, B, A )
  }
```


Time complexity of Tower of Hanoi using Recursion

- For $n > 1$, twice whatever the number of moves required for $n-1$ are made plus the move made by the write statement.
- Hence, the number of moves made by TOWER on input n is:

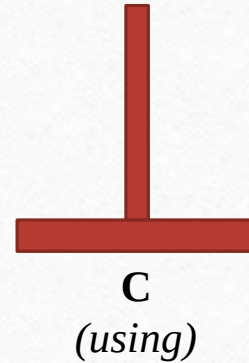
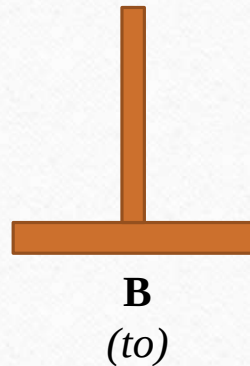
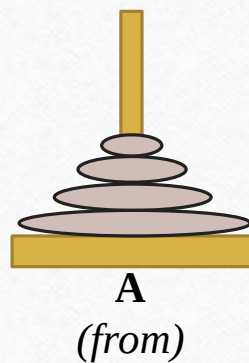
$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n-1) + 1 & \text{otherwise} \end{cases}$$

For $n \geq 1$, TOWER(n, A, B, C) makes $2^n - 1$ moves.

The time for TOWER(n, A, B, C) is $\Theta(2^n)$ because it is proportional to the number of moves made

Question Time

- Ques: Can you guess minimum time required to solve a tower of Hanoi puzzle for $n=4$ disks, assuming one move takes 2 seconds?



- Ans: 30 seconds(=15 moves $\times$ 2 seconds)

Thank You